CASE STUDY | Software Full Stack Development Intern application, Infilect

The Rs 10 Packet Problem

ShelfSignal: a new data product hiding in shelf photos, built as a working prototype

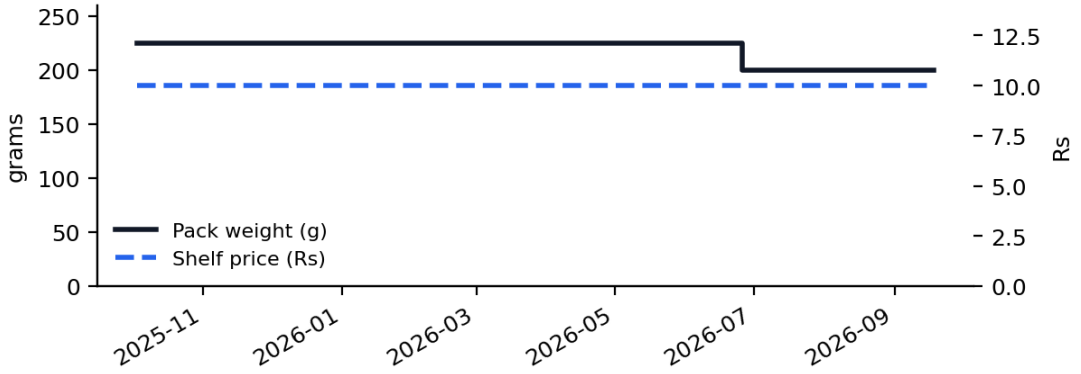
Anisha Tiwary | Bengaluru | github.com/Anisha-Tiwaryy/shelfsignal

The idea in one line: every shelf photo Infilect processes already records a SKU, a shelf price and a pack weight. Put millions of those rows side by side across brands, stores and months, and they reveal things no single brand's dashboard shows: which products are quietly shrinking, how fast new launches really spread, and how much inflation is hiding inside smaller packs. Infilect already sells tools. This would be a data product.

1. What I noticed

A Rs 10 packet of biscuits cannot get more expensive. The price is printed on the pack, shoppers know it by heart, and a Rs 11 packet would feel wrong. So when input costs rise, the price stays and the pack gets smaller. Indian business press has been reporting exactly this across FMCG in the last two years, with brands shrinking low-price packs to hold their price points.

That creates a blind spot. Anyone tracking shelf prices sees a flat line. The shopper is paying more per gram. The only record of the change is the weight printed on the pack, and the only people photographing that pack in thousands of stores every week are field teams using image recognition.



One Rs 10 biscuit pack in one store (simulated): price never moves, the pack drops from 225g to 200g, price per gram rises 12.5%.

2. Three signals nobody is selling yet

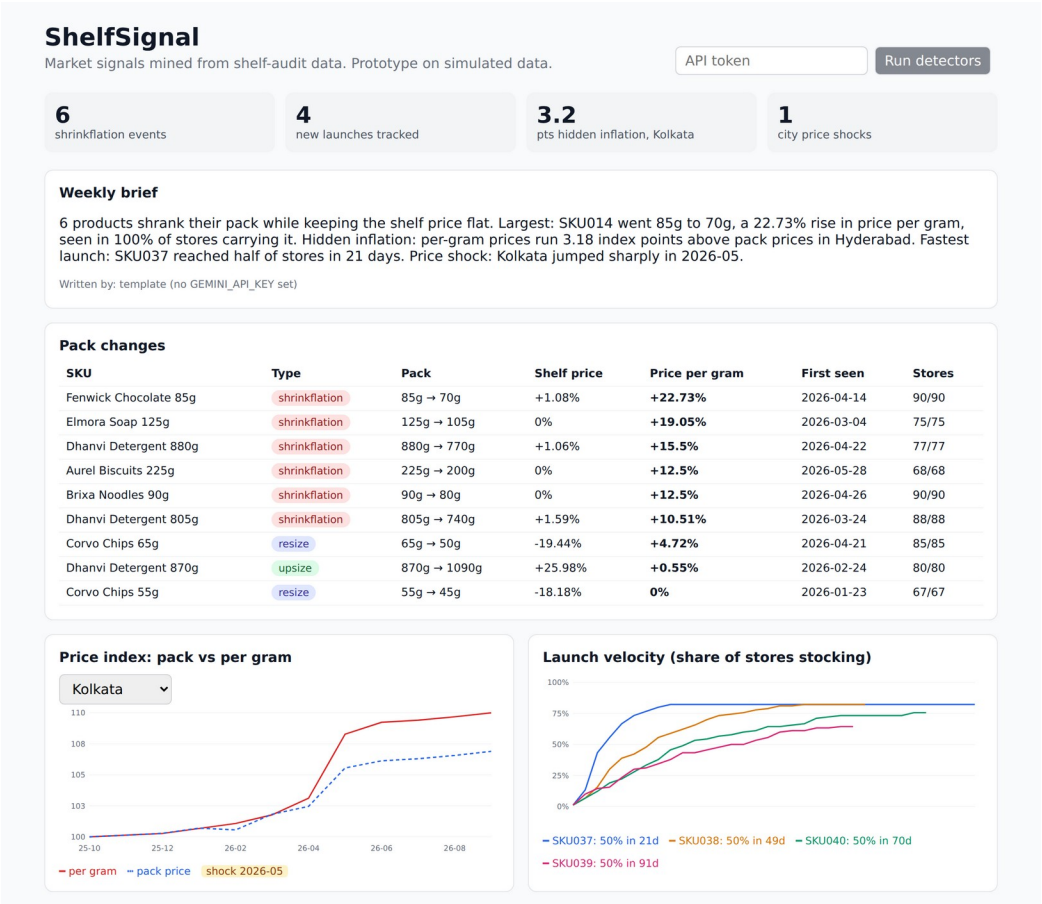
Signal	What it shows	Who would pay for it (hypothesis)
Shrinkflation watch	Which SKUs shrank while the shelf price held, where, since when, in what share of stores	Competing brands' strategy teams; FMCG equity analysts; consumer-protection and media
Launch velocity	Days for a new product to reach 25% and 50% of stores, by city and channel	Brand teams benchmarking their own launch against rivals; distributors
Hidden inflation index	Pack-price index vs price-per-gram index per city, plus sudden city price shocks	Economists, investors, retailers planning price strategy

These are hypotheses to validate, not claims. The point is that the raw material already exists inside the image pipeline and is currently used to answer one client's questions about its own stores.

3. What I built

A full stack prototype in the same stack as the internship role, running on simulated shelf-audit data with known events planted in it, so every detector can be scored against the truth instead of eyeballed.

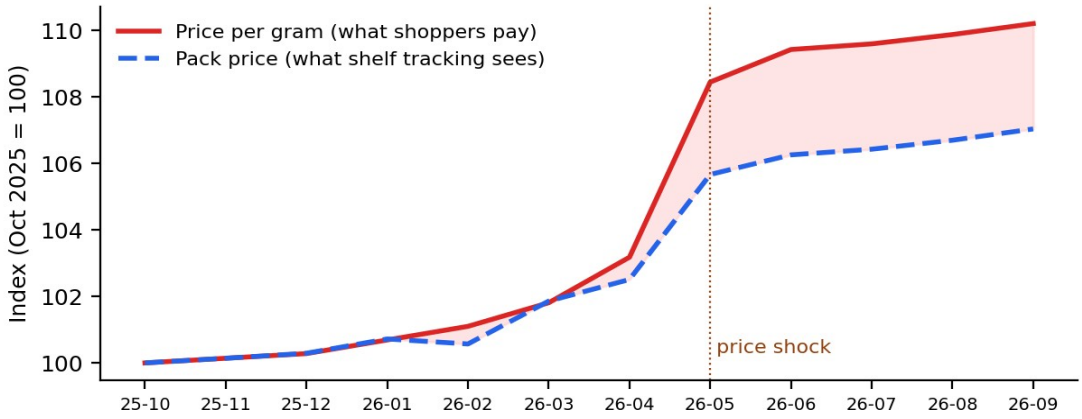
Layer	What it does
Data generator	90 stores in 5 cities, 40 SKUs, 12 months, 78,028 shelf observations. Plants 6 shrinkflation events (2 on fixed Rs 10/20 packs), 2 proportional resizes, 1 upsize, 2 plain price hikes, 4 launches, 1 city price shock, and 1.5% OCR misreads of pack weight
Django + DRF + PostgreSQL	Models for stores, SKUs, observations and signals; 8 REST endpoints; token auth on writes; filters and pagination
Celery + Redis	POST /api/runs/ returns 202 and queues a background detector run; the dashboard polls its status. Full scan of 78,028 rows: about 1.5 seconds
Detectors (pandas)	Pack-change detection with a persistence filter that ignores one-off OCR misreads; launch adoption curves; Jevons price index on pack price and on price per gram; shock detection
GenAI brief	Sends only detector numbers (no images, no store names) to the Gemini API to write a weekly brief; falls back to a template when no key is set and says which one wrote it
React + TypeScript	Dashboard with KPIs, pack-change table, index chart and launch curves; typed API client
Quality	18 pytest tests (unit + API integration on Postgres), GitHub Actions CI with a Postgres service, stress-test script



The React dashboard running against the Django API after a background detector run.

4. Did it work?

Check against planted truth	Result
Shrinkflation events found	6 of 6, zero false positives
Pack changes classified correctly (shrinkflation vs resize vs upsize)	9 of 9
Plain price hikes wrongly flagged as pack changes	0 of 2
City price shock (Kolkata, May 2026)	Found, no false alarms in other cities
Launch speed (days to reach 50% of stores)	About 10 days late for 3 launches, 6 early for 1; lag comes from 14-day visit gaps



Kolkata: what shoppers pay per gram (red) pulls away from what pack-price tracking sees (blue). Every city ends about 3 index points apart.

5. Where it breaks, and what the real version needs

I stress-tested the detectors across 3 random seeds per setting instead of reporting only the clean run:

Setting	Shrinkflation recall	Precision	False pack signals	Launch error
10 to 90 stores	1.0	1.0	0	5 to 7 days
OCR misreads 1.5% to 10%	1.0	1.0	0	7 to 8 days
OCR misreads 20%	1.0	0.92	1.7 per run	5 days
Visits every 28 days	1.0	0.95	0	16 days

- **OCR quality is the ceiling.** At 20% misreads, repeated errors survive the persistence filter. In production the pack-weight read would come from Infilect's own vision models, so its confidence score should gate what reaches the detectors.
- **Data rights come first.** Images belong to client relationships. A product like this only works on aggregated signals, with minimum-store thresholds so no single client's data can be reverse engineered, and with client opt-in.
- **Coverage bias.** Stores visited for one brand are not a random sample of the market; indices need weighting by channel and city.
- **My own blind spot.** The detectors were scored on a world I designed. Real shelves will be messier: relabelled SKUs, regional pack variants, partial shelf photos.

6. What I would do as an intern at Infilect

- **Week 1 to 4:** learn the Django/DRF codebase and ship small features and bug fixes with tests, the core of the role.
- **Reusable synthetic shelf fixtures.** The generator I wrote plants known events on purpose. The same idea gives engineering a realistic, labelled test dataset for API and pipeline integration tests without touching client data.
- **A pack-weight confidence check.** A small background job that flags SKUs whose recognised pack weight flips between visits, catching OCR drift before it reaches client reports.
- **A ShelfSignal pilot, if the team finds it worth exploring.** One category, aggregated data, one internal report, and an honest read on whether the signals hold up on real shelves.

Honest limits

All data here is simulated and every brand and SKU name is invented. Figures describe how the prototype performs on that data, not facts about any real market. The Gemini path is wired but was not run (no API key in the build environment); the brief in the screenshot comes from the template fallback, which the app labels as such.

Code, tests and the stress test: github.com/Anisha-Tiwaryy/shelfsignal